Project 2: Train Your Own GPT

May 2026

1 Introduction

The era of large language models (LLMs) has transformed natural language processing, with GPT-style models demonstrating remarkable abilities in text generation and understanding. This assignment aims to provide hands-on experience with the full lifecycle of LLM development: from implementing a modern Transformer architecture, to exploring scaling laws and architectural innovations, and finally fine-tuning a pre-trained model for practical use.

Important Note:

- The content in this document and the provided starter code are for reference only. You are encouraged to independently explore implementations, libraries, and training strategies to complete the requirements. There is no single "correct" solution—focus on understanding the principles and justifying your design choices.
- Large language modeling is a rapidly evolving field, and some concepts mentioned in this assignment may not have been covered in detail in lectures. An important objective of this project is to develop your ability for independent learning and research. You are expected to proactively consult relevant literature and resources to master unfamiliar concepts.

2 Part A: Implementing a GPT with Transformer + RoPE

Background: The Transformer architecture, with its self-attention mechanism, is the foundation of modern LLMs. Rotary Positional Embedding (RoPE) is a widely used enhancement that injects position information into the attention mechanism in a natural way. In this part, you will implement a GPT-style autoregressive language model from scratch, train it on a small dataset, and tune its hyperparameters.

2.1 Dataset and Starter Code

We use the Penn Treebank dataset, a standard corpus for language modeling, available via the course website. It contains approximately 929,000 training tokens from Wall Street Journal articles, split into training and validation sets. The dataset is designed to be efficient enough that a single training run takes about 10–20 minutes on a single GPU.
The course materials include the following directory structure:

- `"./src/"` contains basic starter code to help you get started
- `"./data/"` contains the training and validation datasets

You will use this same dataset for both Part A and Part B of the assignment.

2.2 Requirements

1. **Model Implementation:** Manually implement a Transformer decoder with RoPE. Key components include:

   - Token embedding layer
   - Rotary Positional Embedding (RoPE)
   - Multi-head self-attention (with causal masking)
   - SwiGLU feed-forward network (FFN)
   - Pre-norm Layer normalization and residual connections
   - Final linear head for next-token prediction

   You may use basic PyTorch modules (e.g., `nn.Linear`, `nn.LayerNorm`) but should implement the core logic (attention, RoPE, training loop) yourself. The starter code is provided for reference.

2. **Training:** Train your model from scratch on the provided dataset, using the next word prediction target with cross entropy loss.

3. **Hyperparameter Tuning:** Experiment with hyperparameters (e.g., learning rate, batch size, number of layers, number of heads, embedding dimension) to find a "good" setting. Use validation loss/perplexity as your metric. Plot the training/validation loss curves for your best run.

Report Requirement: In your report, describe your model implementation details, training process, hyperparameter search space and results, and present the loss curves for your best-performing model.

3 Part B: Scaling Laws and Architectural Study

In this part, you will investigate how model performance scales with parameters and context, and how simple architectural changes affect performance. Use the same Penn Treebank dataset as in Part A.

3.1 Scaling Laws

Study the following two fundamental relationships in language modeling:

1. **Loss vs. Non-Embedding Parameters:** Investigate how validation loss scales with model size, specifically measured by the *number of parameters excluding the token embedding layer*. Keep the dataset and total training steps fixed, and vary the model size (e.g., by changing the number of layers, heads, or embedding dimension). Plot validation loss against the count of non-embedding parameters on a log-log scale. **Fit a linear function to your data points**—you should observe that the relationship is approximately linear in log-log space, consistent with the power-law scaling behavior reported in the literature.

2. **Loss vs. Token Position (Context Length Effect):** For a single well-trained model, investigate how the training loss varies depending on the position of the token in the sequence. Specifically, group the tokens by their position in the context window (e.g., positions 1–32, 33–64, 65–96, etc.) and compute the average loss for tokens in each group. Plot this average loss against the position group. This reveals how prediction difficulty changes as the model has access to more context.

3.2 Architectural Variations

Implement and study the following architectural modifications:

- **QK Norm:** Apply normalization (e.g., LayerNorm) to the query and key vectors before computing attention scores.
- **Attention Gate:** Add a learnable gating mechanism (e.g., a simple sigmoid gate) to the attention output. See the SDPA Elementwise in https://arxiv.org/abs/2505.06708.
- **Value Embedding:** Modify the value tokens by:

```
value = value + self.value_embedding(input_ids)
```

Train these modified architectures with varying sizes (different non-embedding parameter counts). Plot all your results (baseline from Part A and the new architectures) on the same *Loss vs. Non-Embedding Parameters* graph. This allows for a fair comparison of the different architectures across scales. Briefly discuss your findings and explain why certain architectures perform better or worse than the baseline.
Report Requirement: In your report, present both scaling law plots with the linear fit for the parameter scaling curve, show the architectural comparison plot, and analyze the trends you observe in both experiments.

4 Part C: Fine-Tuning a Pre-Trained Model

Modern LLMs benefit greatly from pre-training on massive general-purpose datasets. In this part, you will take a small pre-trained model and fine-tune it on a domain you are familiar with, creating a model that can actually "talk" coherently in that domain.

4.1 Setup

- **Pre-trained Model:** Choose a small open-source pre-trained model (e.g., Qwen-0.5B, TinyLLaMA, or a small GPT-2 variant). You may use libraries like Hugging Face `transformers` to load the pre-trained weights and tokenizer.
- **Domain Dataset:** Pick a domain you know well (e.g., your research field, a hobby, a specific writing style). Collect or find a small dataset in this domain (it does not need to be huge—even a few thousand tokens can produce noticeable results).

4.2 Fine-Tuning and Observation

1. **Fine-Tuning:** Fine-tune the pre-trained model on your domain-specific dataset using the standard next-token prediction objective. You do not need to train for many tokens—fine-tuning is usually fast and overfitting can occur quickly. Monitor the training loss to determine when to stop.
2. **Generation and Observation:** After fine-tuning, evaluate your model qualitatively by generating text using prompts relevant to your domain. Compare the outputs of the fine-tuned model with the original base model. Do you see an improvement in coherence, relevance, and domain-specific knowledge? Provide a few interesting examples of generated text in your report and discuss the differences you observe.

Report Requirement: In your report, describe your chosen domain and dataset, your fine-tuning setup and process, and present a side-by-side comparison of generated text from the base model and your fine-tuned model with analysis.

5 Bonus (Optional): Diffusion Language Model

(This task is optional and for the curious, going beyond the core assignment. This task is also relatively free-form, you can choose to study any of the proposed topics, or discuss with the TAs if you want to explore different dimensions of DLMs.) While standard autoregressive models (like the GPT you just trained) dominate natural language processing, a fascinating alternative paradigm is rapidly gaining traction: *Diffusion Language Models*. In this open-ended task, you are encouraged to explore how diffusion processes—which have revolutionized image and video generation—can be adapted for discrete text generation.

5.1 What is a Diffusion Language Model?

Standard GPT models generate text strictly left-to-right, predicting one token at a time. In contrast, diffusion models typically start with a sequence of pure noise (e.g., entirely random tokens, continuous noise in the embedding space, or fully masked sequences) and gradually denoise it to produce a coherent sequence all at once. Because text is inherently discrete, applying the continuous noise processes used in image generation is non-trivial.
Researchers have developed various methods to bridge this gap, exploring different generative paradigms:

- **Continuous Diffusion:** Applying continuous noise directly to word embeddings (e.g., Diffusion-LM).
- **Masked/Absorbing State Diffusion:** Using discrete categorical diffusion where tokens gradually unmask over time (e.g., LLaDA).
- **Uniform-State and The Diffusion Duality:** A recent breakthrough demonstrates that uniform-state discrete diffusion processes naturally emerge from an underlying continuous Gaussian diffusion. This "Diffusion Duality" enables the transfer of powerful techniques from continuous diffusion (like consistency distillation for rapid, few-step generation) directly into discrete language modeling (e.g., the Duo framework).

This overarching approach offers exciting potential advantages, including bidirectional context during generation, controllable text editing, and highly flexible decoding strategies.

5.2 Some suggestions

This is an open-ended exploration, and you are free to define your own scope. Here are a few directions you might consider. If you wish to study other topics on diffusion language models, please discuss with the TAs.

- **Comparative Experiment:** Train a simple diffusion language model from scratch (e.g., using a masked absorbing-state diffusion approach) and compare its performance, generation quality, and inference dynamics against the GPT model you trained in the core project.

- **Paradigm Exploration:** Research and systematically compare different modern DLM frameworks (like masked diffusion vs. uniform-state diffusion). How do their forward/reverse processes and loss formulations differ?
- **AR to Diffusion Adaptation:** Investigate how a standard pre-trained autoregressive model can be fine-tuned or adapted into a diffusion-based or masked generative model.
- **Theory & Validation:** Dive deep into the math. Explore the theoretical duality between different generative paradigms (such as how discrete diffusion emerges from Gaussian diffusion). Derive the training objective for a specific discrete diffusion process, and validate your derivations with a small-scale toy training experiment.

Describe the specific approach, codebase, or papers you investigated. If you run code, please detail your experimental setup (model architecture, noise schedule, dataset, training duration) and analyze the outcomes (e.g., qualitative examples of generated text, learning curves, failure cases).

5.3 References and Open-Source Codebases

To help you get started without building everything from scratch, here are a few pioneering papers and their corresponding open-source repositories. You are highly encouraged to explore these resources:

- **Theoretical Foundations & Duality:**
  The Diffusion Duality (Sahoo et al., ICML 2025). This paper establishes a theoretical connection demonstrating that uniform-state discrete diffusion emerges from continuous Gaussian diffusion. It introduces the Duo framework, enabling advanced techniques like curriculum learning and discrete consistency distillation for fast text generation.
  Codebase: https://github.com/s-sahoo/duo
- **Continuous Diffusion for Text:**
  Diffusion-LM Improves Controllable Text Generation (Li et al., NeurIPS 2022). This paper introduces a continuous diffusion model applied directly to word embeddings.
  Codebase: https://github.com/XiangLi1999/Diffusion-LM
- **Masked/Discrete Diffusion for Large Language Models:**
  Large Language Diffusion Models (LLaDA) (Nie et al., 2025). This recent work scales up masked diffusion to the billion-parameter regime, showing performance competitive with standard AR models like LLaMA.
  Codebase: https://github.com/ML-GSAI/LLaDA